Aula 09 — Listas e vetores em Python

Guardando vários valores em uma única variável

Dieimes Nunes

Retomando a Aula 08

Na aula anterior, usamos repetições para trabalhar com vários dados.

Por exemplo:

- receber várias notas;
- somar valores;
- contar aprovados;
- validar entradas;
- gerar relatórios.

Mas havia uma limitação importante:

> Depois que o programa usava uma nota, ela era substituída pela próxima.

Hoje vamos aprender a guardar vários valores.

O problema da aula

Imagine que precisamos receber as notas de cinco alunos.

Poderíamos criar várias variáveis:

```
nota1 = 8
nota2 = 6
nota3 = 9
nota4 = 7
nota5 = 5
```

Isso funciona para poucos valores.

Mas e se forem 30 alunos?

E se forem 100 produtos?

Precisamos de uma estrutura melhor.

Lista como vetor

Em Python, usamos **listas** para representar vetores.

Uma lista permite guardar vários valores dentro de uma única variável.

Exemplo:

```
notas = [8, 6, 9, 7, 5]
```

Aqui temos:

- uma variável chamada `notas`;
- cinco valores guardados dentro dela;
- cada valor ocupa uma posição.

Pensando em uma lista

Podemos imaginar a lista como uma sequência de caixas.

```
notas = [8, 6, 9, 7, 5]
          0  1  2  3  4
```

Cada valor possui um **índice**.

Em Python, os índices começam em zero.

Isso é muito importante:

```
primeiro elemento → índice 0
segundo elemento  → índice 1
terceiro elemento → índice 2
```

Atividade 1 — Entendendo índices

Observe a lista:

```
nomes = ["Ana", "Bruno", "Carla", "Diego", "Eduarda"]
```

No caderno, responda:

1. Qual nome está no índice `0`?
2. Qual nome está no índice `2`?
3. Qual nome está no índice `4`?
4. Quantos elementos existem na lista?
5. Qual é o índice do último elemento?

Depois confira no Python.

Acessando elementos da lista

Para acessar um elemento, usamos o nome da lista e o índice entre colchetes.

```
nomes = ["Ana", "Bruno", "Carla"]

print(nomes[0])
print(nomes[1])
print(nomes[2])
```

Saída:

```
Ana
Bruno
Carla
```

Se tentar acessar um índice que não existe, ocorrerá erro.

Erro de índice

Veja o exemplo:

```
nomes = ["Ana", "Bruno", "Carla"]

print(nomes[3])
```

A lista possui os índices:

`0, 1, 2`

O índice `3` não existe.

Esse erro é comum no início.

Criando listas

Podemos criar uma lista já com valores:

```
notas = [8.5, 7.0, 9.2]
```

Também podemos criar uma lista vazia:

```
notas = []
```

A lista vazia é muito usada quando vamos preencher os dados durante a execução do programa.

Adicionando valores com append()

Para adicionar um valor ao final da lista, usamos `.append()`.

```
nomes = []

nomes.append("Ana")
nomes.append("Bruno")
nomes.append("Carla")

print(nomes)
```

Saída:

```
['Ana', 'Bruno', 'Carla']
```

Atividade 2 — Criando uma lista de nomes

Crie um arquivo chamado:

`lista_nomes.py`

O programa deve:

1. criar uma lista vazia chamada `nomes`;
2. adicionar três nomes usando `.append()`;
3. mostrar a lista completa;
4. mostrar cada nome separadamente usando índice.

Depois, altere o programa para cadastrar cinco nomes.

Lendo dados para uma lista

Podemos combinar lista com `for`.

```
nomes = []

for contador in range(5):
    nome = input("Digite um nome: ")
    nomes.append(nome)

print(nomes)
```

A cada repetição:

- o usuário digita um nome;
- o nome é adicionado na lista;
- a lista cresce.

Mostrando os itens da lista

Podemos percorrer a lista usando `for`.

```
nomes = ["Ana", "Bruno", "Carla"]

for nome in nomes:
    print(nome)
```

Esse tipo de `for` percorre elemento por elemento.

Não precisamos informar o índice quando queremos apenas acessar cada valor.

Atividade 3 — Cadastro de alunos

Crie um arquivo chamado:

`cadastro_alunos.py`

O programa deve:

1. criar uma lista vazia chamada `alunos`;
2. perguntar quantos alunos serão cadastrados;
3. receber o nome de cada aluno;
4. adicionar cada nome na lista;
5. mostrar todos os nomes cadastrados no final.

Use `for` e `.append()`.

Tamanho da lista com len()

A função `len()` mostra quantos elementos existem em uma lista.

```
nomes = ["Ana", "Bruno", "Carla"]

print(len(nomes))
```

Resultado:

```
3
```

`len()` é muito útil para:

- saber a quantidade de elementos;
- controlar laços;
- evitar acessar índices inexistentes.

Percorrendo lista com índice

Às vezes, precisamos do índice.

```
nomes = ["Ana", "Bruno", "Carla"]

for indice in range(len(nomes)):
    print(indice, nomes[indice])
```

Saída:

```
0 Ana
1 Bruno
2 Carla
```

Esse formato ajuda quando precisamos mostrar posição, alterar valores ou comparar elementos.

Atividade 4 — Lista numerada

Crie um arquivo chamado:

`lista_numerada.py`

O programa deve:

1. cadastrar cinco produtos em uma lista;
2. mostrar os produtos numerados;
3. exibir também o índice real de cada produto.

Exemplo:

```
Índice 0 - Arroz
Índice 1 - Feijão
Índice 2 - Café
```

Alterando valores da lista

Podemos alterar um elemento usando o índice.

```
produtos = ["Arroz", "Feijão", "Café"]

produtos[1] = "Macarrão"

print(produtos)
```

Saída:

```
['Arroz', 'Macarrão', 'Café']
```

O elemento do índice `1` foi substituído.

Removendo valores da lista

Existem várias formas de remover valores.

Remover pelo valor:

```
produtos = ["Arroz", "Feijão", "Café"]
produtos.remove("Feijão")
print(produtos)
```

Remover pelo índice:

```
produtos = ["Arroz", "Feijão", "Café"]
produtos.pop(1)
print(produtos)
```

Nos dois casos, o item será removido.

Cuidado ao remover

Se tentarmos remover um valor que não existe, o Python gera erro.

```
produtos = ["Arroz", "Feijão", "Café"]
produtos.remove("Leite")
```

Antes de remover, podemos verificar se o valor está na lista:

```
if "Leite" in produtos:
    produtos.remove("Leite")
else:
    print("Produto não encontrado.")
```

Atividade 5 — Alterar e remover produtos

Crie um arquivo chamado:

`editar_produtos.py`

O programa deve começar com esta lista:

```
produtos = ["Arroz", "Feijão", "Café", "Açúcar"]
```

Depois deve:

Verificando se um valor existe

O operador `in` verifica se um valor está dentro da lista.

```
alunos = ["Ana", "Bruno", "Carla"]

print("Ana" in alunos)
print("Diego" in alunos)
```

Saída:

```
True
False
```

Isso será importante para pesquisas em listas.

Exemplo: procurar aluno

```
alunos = ["Ana", "Bruno", "Carla", "Diego"]

nome = input("Digite o nome do aluno: ")

if nome in alunos:
    print("Aluno encontrado.")
else:
    print("Aluno não encontrado.")
```

Esse é um exemplo simples de busca.

Na próxima aula, vamos fazer buscas percorrendo a lista manualmente.

Atividade 6 — Buscar nome

Crie um arquivo chamado:

`buscar_nome.py`

O programa deve:

1. criar uma lista com cinco nomes;
2. pedir um nome para pesquisa;
3. verificar se o nome existe na lista;
4. mostrar uma mensagem de encontrado ou não encontrado.

Depois teste com:

- um nome que existe;
- um nome que não existe;
- letras maiúsculas e minúsculas diferentes.

Lista de notas

Listas também podem guardar números.

```
notas = [8.5, 7.0, 6.5, 9.0]
```

Podemos somar manualmente:

```
soma = 0

for nota in notas:
    soma = soma + nota

media = soma / len(notas)

print(media)
```

Aqui usamos lista, repetição, acumulador e `len()`.

Atividade 7 — Média com lista

Crie um arquivo chamado:

`media_lista.py`

O programa deve:

1. criar uma lista vazia de notas;
2. pedir a quantidade de notas;
3. receber cada nota e adicionar na lista;
4. calcular a média usando `for`;
5. mostrar a lista de notas e a média final.

Desafio: valide as notas entre 0 e 10.

Listas com tipos diferentes

Python permite misturar tipos em uma lista.

```
dados = ["Ana", 17, 8.5, True]
```

Mas, para iniciantes, é melhor evitar misturar tudo sem necessidade.

Prefira listas com dados do mesmo tipo:

```
nomes = ["Ana", "Bruno", "Carla"]
notas = [8.5, 7.0, 9.0]
idades = [16, 17, 18]
```

Isso deixa o programa mais organizado.

Erros comuns com listas

Alguns erros comuns:

1. esquecer que o índice começa em zero;
2. tentar acessar um índice que não existe;
3. usar parênteses no lugar de colchetes;
4. esquecer de usar `.append()` para adicionar;
5. confundir o valor com o índice;
6. remover um item que não existe;
7. alterar a lista errada.

Errar com listas é comum.

Por isso, teste com calma.

Onde estamos no curso

Agora já conseguimos:

- guardar um único valor em uma variável;
- repetir comandos;
- tomar decisões;
- validar dados;
- guardar vários valores em uma lista.

A lista será base para:

- vetores;
- pesquisas;
- ordenação;
- matrizes;
- registros;
- projetos maiores.

Desafio final da aula

Crie um programa chamado:

`lista_compras.py`

O programa deve ter um menu:

```
1 - Adicionar produto
2 - Listar produtos
3 - Remover produto
0 - Sair
```

Regras:

Fechamento

Nesta aula, iniciamos o estudo de listas.

Vimos:

- criação de listas;
- índices;
- acesso a elementos;
- inclusão com `.append()`;
- alteração;
- remoção;
- verificação com `in`;
- uso de `for` com listas.

Na próxima aula, vamos manipular listas com mais profundidade.